

TC 5033 — Deep Learning

Activity 4: Implementing a Transformer-Based Translator

No.	Name	Student ID
1	Joel Arturo Becerril Balderas	A01797427
2	Angel Eduardo Urueta Puello	A01796724
3	Marco Antonio Chávez García	A01797547
4	Efraín Paredes Balgañón	A01351304

Introduction to the Transformer

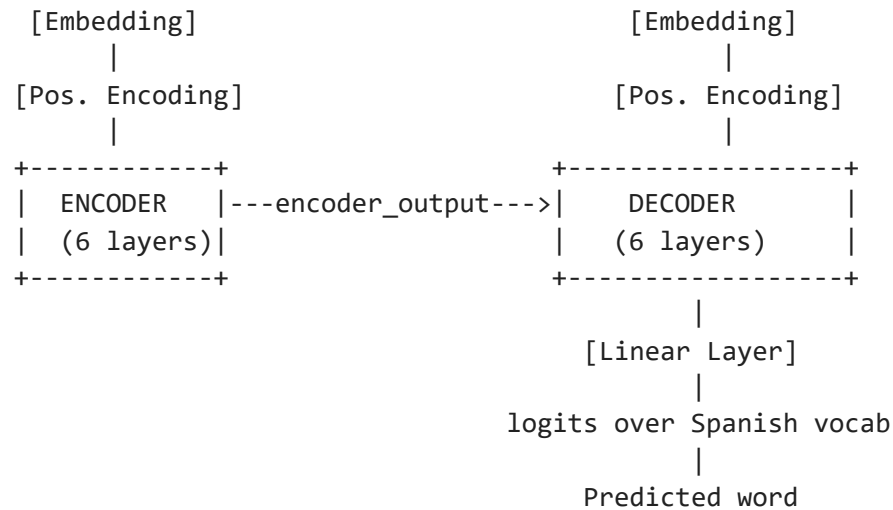
The **Transformer** is a neural network architecture introduced in the paper "*Attention is All You Need*" (Vaswani et al., 2017). Unlike RNNs, it processes the **entire sequence in parallel** using a mechanism called **Self-Attention**, which allows every token to directly interact with every other token regardless of distance.

Why does it matter?

- Foundation of modern LLMs: GPT, BERT, T5, LLaMA, Claude, and beyond.
- Captures **long-range dependencies** that RNNs struggle with.
- Scales efficiently with data and compute.

High-level architecture





In this notebook we build an **English** → **Spanish** translator trained on the [Tatoeba](#) dataset (eng-spa4.txt).

1. Imports and Initial Setup

```

In [1]: # — Core libraries —————
import torch                                # Deep Learning framework
import torch.nn as nn                      # Neural network modules
import torch.nn.functional as F           # Activation functions, softmax,
import torch.optim as optim                # Optimizers (Adam, SGD, etc.)
from torch.utils.data import Dataset, DataLoader # Mini-batch data loading
from collections import Counter            # Word frequency counting
import math                                # sqrt, log
import numpy as np                         # Numerical utilities
import re                                  # Regular expressions for text cl

# Fix the random seed so results are reproducible across runs
import time                                # Execution time tracking

torch.manual_seed(23)
print('Imports complete')
  
```

Imports complete

```

In [2]: # Use CUDA GPU if available; otherwise fall back to CPU.
# Training on a GPU can be 10-50x faster for large models.
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Computing device: {device}')
  
```

Computing device: cuda

```

In [3]: # Maximum number of tokens the model processes per sequence.
# Sequences longer than this are truncated; shorter ones are zero-padded.
MAX_SEQ_LEN = 128
  
```

2. Loading the Dataset

The file `eng-spa4.txt` was prepared from the Tatoeba corpus.

It is a **tab-separated** text file where every line contains one English sentence and its Spanish translation:

```
Go.\tVe.
Hello!\t¡Hola!
I love you.\tTe quiero.
```

File location:

H:\Mi unidad\maestria\IA\Tec\advanced ML\SEm9_9mar\eng-spa4.txt

```
In [4]: # Path to the English-Spanish sentence pairs (Tatoeba dataset)
PATH = '/teamspace/studios/this_studio/eng-spa4.txt'

# Read all lines and split each one into an (English, Spanish) pair
with open(PATH, 'r', encoding='utf-8') as f:
    lines = f.readlines()

# Keep only lines that contain the tab separator; discard malformed lines
eng_spa_pairs = [line.strip().split('\t') for line in lines if '\t' in line]

print(f'Total sentence pairs loaded: {len(eng_spa_pairs):,}')
```

Total sentence pairs loaded: 280,040

```
In [5]: # Quick sanity check – inspect the first five pairs
print('Sample pairs from the dataset:')
for pair in eng_spa_pairs[:5]:
    print(f'  EN: {pair[0]}')
    print(f'  ES: {pair[1]}')
    print()
```

Sample pairs from the dataset:

```
EN: So?
ES: ¿Entonces?
```

```
EN: Go.
ES: Vaya.
```

```
EN: Hi!
ES: ¡Hola!
```

```
EN: Go.
ES: Ve.
```

```
EN: Go.
ES: Vete.
```

```
In [6]: # Separate into individual English and Spanish sentence lists
eng_sentences = [pair[0] for pair in eng_spa_pairs]
spa_sentences = [pair[1] for pair in eng_spa_pairs]

print(f'English sentences : {len(eng_sentences):,}')
print(f'Spanish sentences : {len(spa_sentences):,}')
```

English sentences : 280,040

Spanish sentences : 280,040

3. Transformer Building Blocks

3.1 Positional Embedding

The Transformer has **no recurrence** and processes all tokens simultaneously, so it has no built-in sense of word order.

We fix this by adding a **positional signal** to each token embedding using alternating sine and cosine functions at different frequencies:

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d_{\text{model}})}) \end{aligned}$$

Each position produces a **unique vector** that is added element-wise to the word embedding.

Using sine/cosine (rather than learned embeddings) lets the model generalize to sequence lengths not seen during training.

```
In [7]: class PositionalEmbedding(nn.Module):
        """
        Adds fixed positional information to token embeddings.

        Without this, 'The cat sat' and 'sat cat The' would produce
        identical representations – word order would be invisible to the model.

        Even-indexed dimensions use sine; odd-indexed dimensions use cosine.
        The encoding matrix is computed once at construction time (not learned).
        """
        def __init__(self, d_model, max_seq_len=MAX_SEQ_LEN):
            super().__init__()

            # Pre-compute the full positional encoding matrix: [max_seq_len, d_model]
            self.pos_embed_matrix = torch.zeros(max_seq_len, d_model, device=device)

            # Column vector of position indices: [0, 1, 2, ..., max_seq_len-1]
            token_pos = torch.arange(0, max_seq_len, dtype=torch.float).unsqueeze(1)

            # Frequency scaling term – decreases exponentially with dimension index
            div_term = torch.exp(
                torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
```

```

)

# Fill even columns with sine, odd columns with cosine
self.pos_embed_matrix[:, 0::2] = torch.sin(token_pos * div_term)
self.pos_embed_matrix[:, 1::2] = torch.cos(token_pos * div_term)

# Reshape to [max_seq_len, 1, d_model] so it broadcasts over batch dimension
self.pos_embed_matrix = self.pos_embed_matrix.unsqueeze(0).transpose(0, 1)

def forward(self, x):
    # x shape: [seq_len, batch_size, d_model]
    # Add the positional signal for the first x.size(0) positions
    return x + self.pos_embed_matrix[:x.size(0), :]

```

3.2 Multi-Head Attention

The central mechanism of the Transformer.

Every token computes a weighted sum over **all other tokens** to build a context-aware representation.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Vector	Intuition
Q (Query)	"What information am I looking for?"
K (Key)	"What information do I advertise?"
V (Value)	"What information do I actually provide?"

The dot product $Q \cdot K^T$ measures compatibility; dividing by $\sqrt{d_k}$ prevents gradients from vanishing when d_k is large (large dot products push softmax into saturation).

Why multiple heads?

Running h independent attention operations in parallel lets the model simultaneously capture **different types of relationships** (syntactic structure, semantic similarity, coreference, etc.):

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_o \\ \text{head}_i &= \text{Attention}(Q \cdot W_{qi}, K \cdot W_{ki}, V \cdot W_{vi}) \end{aligned}$$

Split the query, key, and value into multiple heads

to allow the model to focus on different parts of the sentence.

```
In [8]: class MultiHeadAttention(nn.Module):
        """
        Multi-Head Attention module.

        Runs h independent scaled dot-product attention operations in parallel,
        then concatenates and projects the results.

        Args:
            d_model : total embedding dimension (e.g. 512)
            num_heads : number of parallel attention heads (e.g. 8)
        """
        def __init__(self, d_model=512, num_heads=8):
            super().__init__()
            # d_model must split evenly across heads
            assert d_model % num_heads == 0, 'Embedding size not compatible with num heads'

            self.d_v = d_model // num_heads # Dimension of each head's Value subspace
            self.d_k = self.d_v             # Dimension of each head's Key/Query subspace
            self.num_heads = num_heads

            # Four Learned Linear projections (no bias by default in PyTorch Linear)
            self.W_q = nn.Linear(d_model, d_model) # Projects all tokens -> Query space
            self.W_k = nn.Linear(d_model, d_model) # Projects all tokens -> Key space
            self.W_v = nn.Linear(d_model, d_model) # Projects all tokens -> Value space
            self.W_o = nn.Linear(d_model, d_model) # Projects concatenated heads -> Output

        def forward(self, Q, K, V, mask=None):
            batch_size = Q.size(0)

            # 1. Project and split into heads
            # [batch, seq, d_model] -> [batch, num_heads, seq, d_k]
            Q = self.W_q(Q).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
            K = self.W_k(K).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
            V = self.W_v(V).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)

            # 2. Compute attention for all heads simultaneously
            weighted_values, attention = self.scale_dot_product(Q, K, V, mask)

            # 3. Concatenate heads and project back to d_model
            # [batch, num_heads, seq, d_k] -> [batch, seq, d_model]
            weighted_values = (
                weighted_values
                .transpose(1, 2)
                .contiguous()
                .view(batch_size, -1, self.num_heads * self.d_k)
            )
            weighted_values = self.W_o(weighted_values)

            return weighted_values, attention
```

```
def scale_dot_product(self, Q, K, V, mask=None):
    """
    Scaled Dot-Product Attention (core computation).

    scores = (Q · K^T) / sqrt(d_k)    <- scale prevents softmax saturation
    weights = softmax(scores + mask)   <- mask sets unwanted positions to -
    output = weights · V
    """
    # Dot product between every query and every key
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)

    # Apply mask: positions where mask==0 get -1e9 so softmax -> ~0
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)

    # Softmax normalizes scores into attention weights (sum to 1 per query)
    attention = F.softmax(scores, dim=-1)

    # Weighted sum of value vectors
    weighted_values = torch.matmul(attention, V)

    return weighted_values, attention
```

3.3 Position-wise Feed-Forward Network (FFN)

After attention, each position is independently transformed by a small two-layer MLP:

$$\text{FFN}(x) = \text{ReLU}(x \cdot W1 + b1) \cdot W2 + b2$$

d_model (512) --expand--> d_ff (2048) --compress--> d_model (512)

The 4× expansion in the hidden layer gives the model extra representational capacity to transform token features non-linearly, independent of other positions.

```
In [9]: class PositionFeedForward(nn.Module):
    """
    Position-wise Feed-Forward Network (FFN).

    Applied identically and independently to each position.
    Introduces non-linearity after the attention sublayer.

    Architecture: Linear(d_model -> d_ff) -> ReLU -> Linear(d_ff -> d_model)
    """
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.linear1 = nn.Linear(d_model, d_ff)    # Expand: e.g. 512 -> 2048
        self.linear2 = nn.Linear(d_ff, d_model)    # Compress: e.g. 2048 -> 512

    def forward(self, x):
        # ReLU non-linearity is applied between the two linear transformations
        return self.linear2(F.relu(self.linear1(x)))
```



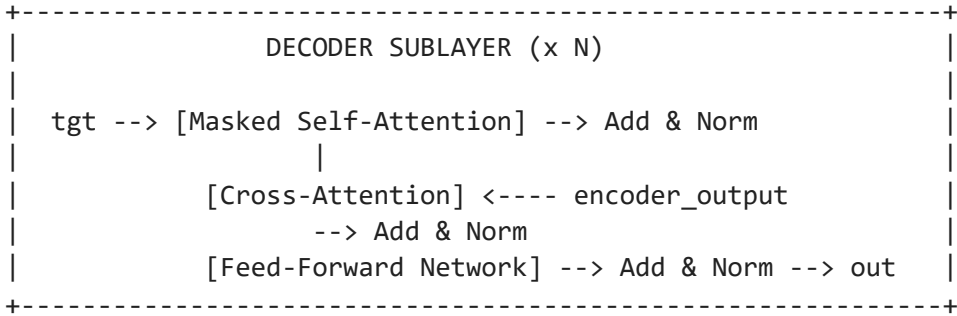
```
class Encoder(nn.Module):
    """
    Full Encoder: N stacked EncoderSubLayer blocks followed by a final LayerNorm.

    Output shape: [batch, src_len, d_model]
    This output is passed to every Decoder layer via Cross-Attention.
    """
    def __init__(self, d_model, num_heads, d_ff, num_layers, dropout=0.1):
        super().__init__()
        # N layers – each has independent learnable parameters
        self.layers = nn.ModuleList(
            [EncoderSubLayer(d_model, num_heads, d_ff, dropout) for _ in range(num_
            )
        ]
        self.norm = nn.LayerNorm(d_model) # Final normalization

    def forward(self, x, mask=None):
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)
```

3.5 Decoder

The Decoder generates the Spanish translation token by token.
Each layer has **three sublayers**:



Sublayer	Key detail
Masked Self-Attention	Causal mask prevents the decoder from seeing future tokens — each position can only attend to itself and earlier positions
Cross-Attention	Q comes from the decoder, K and V come from the encoder output — this is how the model "reads" the English source
Feed-Forward	Same position-wise MLP as in the encoder

```
In [11]: class DecoderSubLayer(nn.Module):
    """
    One Decoder layer.

    Sublayer 1 – Masked Self-Attention:
        The decoder attends to its own previous outputs.
        A causal mask ensures position i can only see positions 0..i.
```

Sublayer 2 – Cross-Attention:

Q = decoder state, K = V = encoder output.
Bridges the source and target languages.

Sublayer 3 – Position-wise FFN.

"""

```
def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
    super().__init__()
    self.self_attn = MultiHeadAttention(d_model, num_heads) # Masked self-a
    self.cross_attn = MultiHeadAttention(d_model, num_heads) # Cross-attenti
    self.feed_forward = PositionFeedForward(d_model, d_ff)
    self.norm1 = nn.LayerNorm(d_model)
    self.norm2 = nn.LayerNorm(d_model)
    self.norm3 = nn.LayerNorm(d_model)
    self.dropout1 = nn.Dropout(dropout)
    self.dropout2 = nn.Dropout(dropout)
    self.dropout3 = nn.Dropout(dropout)

    def forward(self, x, encoder_output, target_mask=None, encoder_mask=None):
        # --- Sublayer 1: Masked Self-Attention ---
        # target_mask prevents attending to future positions (causal mask)
        attn_score, _ = self.self_attn(x, x, x, target_mask)
        x = self.norm1(x + self.dropout1(attn_score))

        # --- Sublayer 2: Cross-Attention ---
        # Q = decoder state (x), K = V = encoder_output (English context)
        enc_attn, _ = self.cross_attn(x, encoder_output, encoder_output, encoder_ma
        x = self.norm2(x + self.dropout2(enc_attn))

        # --- Sublayer 3: Feed-Forward ---
        x = self.norm3(x + self.dropout3(self.feed_forward(x)))
        return x
```

```
class Decoder(nn.Module):
```

"""

Full Decoder: N stacked DecoderSubLayer blocks followed by a final LayerNorm.

Output shape: [batch, tgt_len, d_model]

"""

```
def __init__(self, d_model, num_heads, d_ff, num_layers, dropout=0.1):
    super().__init__()
    self.layers = nn.ModuleList(
        [DecoderSubLayer(d_model, num_heads, d_ff, dropout) for _ in range(num_
        )
    ]
    self.norm = nn.LayerNorm(d_model)

    def forward(self, x, encoder_output, target_mask, encoder_mask):
        for layer in self.layers:
            x = layer(x, encoder_output, target_mask, encoder_mask)
        return self.norm(x)
```

3.6 Full Transformer

The complete model assembles all components and adds two key elements:

Embedding scaling: Embeddings are multiplied by $\sqrt{d_{\text{model}}}$ before positional encoding. This prevents the learned embeddings from being drowned out by the positional signal, since positional values are bounded in $[-1, 1]$ while embeddings can have larger norms.

Masks explained:

Mask	Constructed from	Blocks
source_mask	source != 0	Padding tokens in the English input
target_mask	(target != 0) AND lower-triangular matrix	Padding tokens AND future positions in the Spanish target

```
In [12]: class Transformer(nn.Module):
    """
    Complete Transformer model for English -> Spanish translation.

    Args:
        d_model          : embedding dimension          (512)
        num_heads        : number of attention heads    (8)
        d_ff             : FFN inner dimension         (2048)
        num_layers       : encoder and decoder depth   (6)
        input_vocab_size : English vocabulary size
        target_vocab_size: Spanish vocabulary size
        max_len          : maximum sequence length     (128)
        dropout          : dropout probability         (0.1)
    """
    def __init__(self, d_model, num_heads, d_ff, num_layers,
                  input_vocab_size, target_vocab_size,
                  max_len=MAX_SEQ_LEN, dropout=0.1):
        super().__init__()

        # Learnable embedding tables: integer index -> d_model vector
        self.encoder_embedding = nn.Embedding(input_vocab_size, d_model)
        self.decoder_embedding = nn.Embedding(target_vocab_size, d_model)

        # Fixed positional encoding (shared by encoder and decoder)
        self.pos_embedding = PositionalEmbedding(d_model, max_len)

        # Encoder and Decoder stacks
        self.encoder = Encoder(d_model, num_heads, d_ff, num_layers, dropout)
        self.decoder = Decoder(d_model, num_heads, d_ff, num_layers, dropout)

        # Final linear layer: d_model -> target_vocab_size (raw logits)
        self.output_layer = nn.Linear(d_model, target_vocab_size)

    def forward(self, source, target):
        # Step 1 - Build masks
        source_mask, target_mask = self.mask(source, target)

        # Step 2 - Encode the English source
        # Scale embeddings by sqrt(d_model) as recommended in the original paper
        source = self.encoder_embedding(source) * math.sqrt(self.encoder_embedding.embedding_dim)
        source = self.pos_embedding(source)
```

```

encoder_output = self.encoder(source, source_mask)

# Step 3 – Decode (generate Spanish)
target = self.decoder_embedding(target) * math.sqrt(self.decoder_embedding.
target = self.pos_embedding(target)
output = self.decoder(target, encoder_output, target_mask, source_mask)

# Step 4 – Project to vocabulary logits
return self.output_layer(output)

def mask(self, source, target):
    """
    Build padding and causal masks.

    source_mask [batch, 1, 1, src_len] : True where source token != <pad>
    target_mask [batch, 1, tgt_len, tgt_len] : True where target token != <pad>
                                                AND position is not in the future
    """
    # Encoder padding mask: hide <pad> tokens (index 0)
    source_mask = (source != 0).unsqueeze(1).unsqueeze(2)

    # Decoder padding mask
    target_mask = (target != 0).unsqueeze(1).unsqueeze(2)

    # Causal (look-ahead) mask: Lower-triangular matrix of True
    # Position (i, j) is True only when j <= i – no peeking at the future
    size = target.size(1)
    no_mask = torch.tril(torch.ones((1, size, size), device=device)).bool()

    # Combine: allow only real tokens that are at or before the current position
    target_mask = target_mask & no_mask

    return source_mask, target_mask

```

4. Sanity Check — Shape Verification

Before training on real data we run a single forward pass with random tensors to confirm the model is wired correctly and output shapes match expectations.

```

In [13]: # Tiny vocab and sequence length – just enough to verify shapes
seq_len_source = seq_len_target = 10
batch_size     = 2
v_src = v_tgt  = 50 # Vocabulary size

# Random integer tensors simulating batches of encoded sentences
source_test = torch.randint(1, v_src, (batch_size, seq_len_source))
target_test = torch.randint(1, v_tgt, (batch_size, seq_len_target))

model_test = Transformer(512, 8, 2048, 6, v_src, v_tgt, MAX_SEQ_LEN, 0.1).to(device)
out_test   = model_test(source_test.to(device), target_test.to(device))

# Expected: [batch_size, seq_len_target, target_vocab_size] = [2, 10, 50]

```

```
print(f'Output shape : {out_test.shape}')
print(f'Expected      : [2, 10, 50] --> [batch, tgt_seq_len, vocab_size]')
```

Output shape : torch.Size([2, 10, 50])

Expected : [2, 10, 50] --> [batch, tgt_seq_len, vocab_size]

5. Text Preprocessing

Raw text must be normalized **identically** during training and at inference time. The steps below simplify the vocabulary and ensure consistent tokenization:

Step	Operation	Example
1	Lowercase + strip	Hello! → hello!
2	Collapse spaces	two spaces → two spaces
3	Remove diacritics	está → esta
4	Keep only a-z	hello! → hello
5	Add boundary tokens	hello → <sos> hello <eos>

Why remove diacritics? It reduces vocabulary size and avoids splitting the same word into accented and unaccented variants. The trade-off is that output translations will also lack diacritics.

```
In [14]: def preprocess_sentence(sentence):
        """
        Normalize a raw sentence for the translator.

        This function must be applied consistently to training data,
        validation data, and any sentence passed to translate_sentence().

        Returns a string of the form: '<sos> word1 word2 ... <eos>'
        """
        sentence = sentence.lower().strip()                # Step 1: Lowercase
        sentence = re.sub(r'[" "]+', " ", sentence)        # Step 2: collapse spaces
        sentence = re.sub(r"[á]+", "a", sentence)           # Step 3a: á -> a
        sentence = re.sub(r"[é]+", "e", sentence)           # Step 3b: é -> e
        sentence = re.sub(r"[í]+", "i", sentence)           # Step 3c: í -> i
        sentence = re.sub(r"[ó]+", "o", sentence)           # Step 3d: ó -> o
        sentence = re.sub(r"[ú]+", "u", sentence)           # Step 3e: ú -> u
        sentence = re.sub(r"^[^a-z]+", "", sentence)        # Step 4: remove non-letters
        sentence = sentence.strip()
        sentence = '<sos> ' + sentence + ' <eos>'          # Step 5: add boundary tokens
        return sentence
```

```
In [15]: # Quick demo of the preprocessing pipeline
s1 = '¡Hola @ cómo estás? 123'
```

```
print(f'Original      : "{s1}"')
print(f'Preprocessed: "{preprocess_sentence(s1)}"')
```

Original : "¿Hola @ cómo estás? 123"

Preprocessed: "<sos> hola como estas <eos>"

```
In [16]: # Preprocessing all sentences
start = time.time()

# Apply preprocessing to every sentence in the loaded dataset
eng_sentences = [preprocess_sentence(s) for s in eng_sentences]
spa_sentences = [preprocess_sentence(s) for s in spa_sentences]

# Verify the output on the first three pairs
print('Preprocessed samples:')
for en, es in zip(eng_sentences[:3], spa_sentences[:3]):
    print(f'  EN: {en}')
    print(f'  ES: {es}')
    print()
print(f"Preprocessing done in {time.time() - start:.1f}s")
```

Preprocessed samples:

EN: <sos> so <eos>

ES: <sos> entonces <eos>

EN: <sos> go <eos>

ES: <sos> vaya <eos>

EN: <sos> hi <eos>

ES: <sos> hola <eos>

Preprocessing done in 6.9s

6. Vocabulary

The model works with integers, not strings.

We build a lookup table for each language that maps every word to a unique index:

```
word2idx  : { '<pad>':0, '<unk>':1, 'the':2, 'i':3, ... }
idx2word  : { 0:'<pad>', 1:'<unk>', 2:'the', 3:'i', ... }
```

Words are sorted by **frequency** (most common first) so the most important tokens receive small indices, which slightly improves embedding look-up cache locality.

```
In [17]: def build_vocab(sentences):
        """
        Build word <-> index mappings from a list of preprocessed sentences.

        Special tokens (always assigned fixed indices):
        <pad>  -> 0 : padding token; ignored by CrossEntropyLoss
        <unk>  -> 1 : fallback for words absent from the training vocabulary
```

```

All other words are assigned indices starting from 2,
in descending order of frequency.
"""
# Count every word across all sentences
words = [w for sentence in sentences for w in sentence.split()]
word_count = Counter(words)

# Sort by frequency – most common words get the lowest indices (after 0 and 1)
sorted_words = sorted(word_count.items(), key=lambda x: x[1], reverse=True)

# Assign indices from 2 onwards
word2idx = {word: idx for idx, (word, _) in enumerate(sorted_words, 2)}
word2idx['<pad>'] = 0
word2idx['<unk>'] = 1

# Reverse mapping used during decoding to convert indices back to words
idx2word = {idx: word for word, idx in word2idx.items()}

return word2idx, idx2word

```

```

In [18]: start = time.time()

# Build separate vocabularies for each language
eng_word2idx, eng_idx2word = build_vocab(eng_sentences)
spa_word2idx, spa_idx2word = build_vocab(spa_sentences)

eng_vocab_size = len(eng_word2idx)
spa_vocab_size = len(spa_word2idx)

print(f'English vocabulary : {eng_vocab_size:,} tokens')
print(f'Spanish vocabulary : {spa_vocab_size:,} tokens')
print(f"Vocabulary built in {time.time() - start:.1f}s")

```

English vocabulary : 28,415 tokens
 Spanish vocabulary : 48,285 tokens
 Vocabulary built in 1.1s

7. Dataset and DataLoader

We wrap the data in a standard PyTorch `Dataset` so the `DataLoader` can:

- Deliver examples in **mini-batches** (more efficient GPU utilization than sample-by-sample)
- **Shuffle** the dataset each epoch (breaks spurious correlations from ordering)
- Apply **dynamic padding** via `collate_fn` so every tensor in a batch is the same length

```

In [19]: class EngSpaDataset(Dataset):
        """
        PyTorch Dataset for English-Spanish sentence pairs.

        __getitem__ returns two integer tensors:

```

```

    eng_idx : token index sequence for the English sentence
    spa_idx : token index sequence for the Spanish sentence
    """
    def __init__(self, eng_sentences, spa_sentences, eng_word2idx, spa_word2idx):
        self.eng_sentences = eng_sentences
        self.spa_sentences = spa_sentences
        self.eng_word2idx = eng_word2idx
        self.spa_word2idx = spa_word2idx

    def __len__(self):
        return len(self.eng_sentences)

    def __getitem__(self, idx):
        # Convert each word to its vocab index; unknown words fall back to <unk>
        eng_idx = [
            self.eng_word2idx.get(w, self.eng_word2idx['<unk>'])
            for w in self.eng_sentences[idx].split()
        ]
        spa_idx = [
            self.spa_word2idx.get(w, self.spa_word2idx['<unk>'])
            for w in self.spa_sentences[idx].split()
        ]
        return torch.tensor(eng_idx), torch.tensor(spa_idx)

```

```

In [20]: def collate_fn(batch):
    """
    Collate a list of (eng_tensor, spa_tensor) pairs into padded batches.

    Problem : variable-length sequences cannot be stacked directly.
    Solution : truncate to MAX_SEQ_LEN, then right-pad with 0 (<pad>)
               so all tensors in the batch share the same shape.
    """
    eng_batch, spa_batch = zip(*batch)

    # Truncate sequences that exceed the maximum allowed length
    eng_batch = [seq[:MAX_SEQ_LEN].clone().detach() for seq in eng_batch]
    spa_batch = [seq[:MAX_SEQ_LEN].clone().detach() for seq in spa_batch]

    # Pad to the length of the longest sequence in this batch
    eng_batch = torch.nn.utils.rnn.pad_sequence(
        eng_batch, batch_first=True, padding_value=0)
    spa_batch = torch.nn.utils.rnn.pad_sequence(
        spa_batch, batch_first=True, padding_value=0)

    return eng_batch, spa_batch

```

8. Training Loop

Teacher Forcing

During training we use **teacher forcing**: the ground-truth target tokens are fed directly as decoder input rather than the model's own previous predictions.

This prevents compounding errors from early wrong predictions and stabilizes training:

```
Full target : <sos> hola mundo <eos>
Decoder in  : <sos> hola mundo          <-- all tokens except
the last
Expected out :          hola mundo <eos> <-- all tokens except
<sos>
```

At each position the model learns to predict the **next token** given all prior tokens.

```
In [21]: def train(model, dataloader, loss_function, optimiser, epochs):
        """
        Standard training loop for the Transformer.

        For each epoch:
        1. Iterate over mini-batches
        2. Shift target for teacher forcing
        3. Forward pass -> compute logits
        4. CrossEntropy loss (ignores <pad> tokens)
        5. Backpropagation -> compute gradients
        6. Adam step -> update parameters
        """
        model.train() # Activates dropout and other training-specific behaviors
        total_start = time.time()

        for epoch in range(epochs):
            epoch_start = time.time()
            total_loss = 0

            for eng_batch, spa_batch in dataloader:
                eng_batch = eng_batch.to(device)
                spa_batch = spa_batch.to(device)

                # Teacher forcing: shift the target by one position
                target_input = spa_batch[:, :-1] # Drop Last
                target_output = spa_batch[:, 1:].contiguous().view(-1) # Drop first

                optimiser.zero_grad() # Clear old

                output = model(eng_batch, target_input) # Forward pa
                output = output.view(-1, output.size(-1)) # Flatten fo

                loss = loss_function(output, target_output) # Compute Lo
                loss.backward() # Backpropag
                optimiser.step() # Update wei

                total_loss += loss.item()

            avg_loss = total_loss / len(dataloader)
            epoch_time = time.time() - epoch_start
            elapsed = time.time() - total_start
```

```

    remaining = (elapsed / (epoch + 1)) * (epochs - epoch - 1)
    print(f'Epoch {epoch+1:>2}/{epochs} | Loss: {avg_loss:.4f} | {epoch_tim

```

```

In [22]: BATCH_SIZE = 64

dataset      = EngSpaDataset(eng_sentences, spa_sentences, eng_word2idx, spa_word2idx)
dataloader   = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, collate_fn=collate_fn)

print(f'Dataset size      : {len(dataset):,} pairs')
print(f'Batches per epoch : {len(dataloader):,}')

```

```

Dataset size      : 280,040 pairs
Batches per epoch : 4376

```

```

In [23]: # Instantiate the Transformer with the hyperparameters from the original paper
model = Transformer(
    d_model      = 512,
    num_heads    = 8,
    d_ff         = 2048,
    num_layers   = 6,
    input_vocab_size = eng_vocab_size,
    target_vocab_size = spa_vocab_size,
    max_len      = MAX_SEQ_LEN,
    dropout      = 0.1
).to(device)

# CrossEntropyLoss: measures prediction error; ignore_index=0 skips <pad> tokens
loss_function = nn.CrossEntropyLoss(ignore_index=0)

# Adam: adaptive per-parameter Learning rate – standard choice for Transformers
optimiser = optim.Adam(model.parameters(), lr=0.0001)

total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'Trainable parameters: {total_params:,}')

```

```

Trainable parameters: 108,181,149

```

```

In [24]: # Approximate training times:
# GPU (NVIDIA T4) : ~5-10 min per epoch
# CPU : ~30-60 min per epoch

train_start = time.time()
print("Starting training...\n")

train(model, dataloader, loss_function, optimiser, epochs=10)

total_time = time.time() - train_start
print(f"\n=====")
print(f"Training complete! Total: {total_time/60:.1f} min ({total_time:.0f}s)")
print(f"=====")

```

Starting training...

Epoch	1/10		Loss: 3.5558		962s/epoch		Elapsed: 16.0min		ETA: 144.3min
Epoch	2/10		Loss: 2.1675		961s/epoch		Elapsed: 32.0min		ETA: 128.2min
Epoch	3/10		Loss: 1.6749		964s/epoch		Elapsed: 48.1min		ETA: 112.2min
Epoch	4/10		Loss: 1.3544		963s/epoch		Elapsed: 64.2min		ETA: 96.2min
Epoch	5/10		Loss: 1.1097		965s/epoch		Elapsed: 80.2min		ETA: 80.2min
Epoch	6/10		Loss: 0.9127		964s/epoch		Elapsed: 96.3min		ETA: 64.2min
Epoch	7/10		Loss: 0.7531		962s/epoch		Elapsed: 112.3min		ETA: 48.1min
Epoch	8/10		Loss: 0.6280		961s/epoch		Elapsed: 128.4min		ETA: 32.1min
Epoch	9/10		Loss: 0.5360		960s/epoch		Elapsed: 144.4min		ETA: 16.0min
Epoch	10/10		Loss: 0.4698		963s/epoch		Elapsed: 160.4min		ETA: 0.0min

```
=====
Training complete! Total: 160.4 min (9625s)
=====
```

9. Translation — Greedy Decoding

At inference time the translation is unknown, so we generate tokens **auto-regressively**:

```
Step 0  decoder input : [<sos>]                -> predict w1
Step 1  decoder input : [<sos>, w1]             -> predict w2
Step 2  decoder input : [<sos>, w1, w2]         -> predict w3
...
Step k  decoder input : [<sos>, w1, ..., w_{k-1}] -> predict <eos>
(stop)
```

At each step we choose the **single highest-probability token** (greedy).

This is simple and fast, but does not guarantee the globally best sequence. *Beam search* ($k \geq 2$) would keep the top-k candidates alive at each step and typically produces better translations.

```
In [25]: def sentence_to_indices(sentence, word2idx):
        """Convert a preprocessed sentence string to a list of vocabulary indices."""
        return [word2idx.get(w, word2idx['<unk>']) for w in sentence.split()]

def indices_to_sentence(indices, idx2word):
    """Convert a list of indices to a readable string, skipping <pad> tokens."""
    return ' '.join(
        idx2word[i] for i in indices
        if i in idx2word and idx2word[i] != '<pad>'
    )

def translate_sentence(model, sentence, eng_word2idx, spa_idx2word,
                      max_len=MAX_SEQ_LEN, device='cpu'):
    """
    Translate one English sentence to Spanish using greedy decoding.
```

```

Args:
    model          : trained Transformer
    sentence       : raw English string (preprocessed internally)
    eng_word2idx   : English word -> index mapping
    spa_idx2word   : Spanish index -> word mapping
    max_len        : maximum tokens to generate before forcing a stop
    device         : 'cpu' or 'cuda'

Returns:
    Translated sentence as a plain string.
    """
    model.eval() # Disable dropout for deterministic inference

    # Preprocess with the same pipeline used at training time
    sentence = preprocess_sentence(sentence)
    input_tensor = torch.tensor(
        sentence_to_indices(sentence, eng_word2idx)
    ).unsqueeze(0).to(device)

    # Start the decoder output sequence with the <sos> token
    tgt_indices = [spa_word2idx['<sos>']]
    tgt_tensor = torch.tensor(tgt_indices).unsqueeze(0).to(device)

    with torch.no_grad(): # No gradient tracking needed at inference time
        for _ in range(max_len):
            output = model(input_tensor, tgt_tensor).squeeze(0)

            # Greedy: take the highest-logit token at the last time step
            next_token = output.argmax(dim=-1)[-1].item()

            tgt_indices.append(next_token)
            tgt_tensor = torch.tensor(tgt_indices).unsqueeze(0).to(device)

            if next_token == spa_word2idx['<eos>']: # Stop at end-of-sequence
                break

    return indices_to_sentence(tgt_indices, spa_idx2word)

```

```

In [26]: def evaluate_translations(model, sentences, eng_word2idx, spa_idx2word,
                                max_len=MAX_SEQ_LEN, device='cpu'):
    """Translate and print results for a list of English test sentences."""
    print('=' * 60)
    print('                TRANSLATION RESULTS')
    print('=' * 60)
    for i, sentence in enumerate(sentences, 1):
        translation = translate_sentence(
            model, sentence, eng_word2idx, spa_idx2word, max_len, device
        )
        print(f'[{i:02d}] EN: {sentence}')
        print(f'      ES: {translation}')
        print('-' * 60)

```

9.1 Test sentences

We test with 13 sentences spanning different difficulty levels, verb tenses, and topics.

```
In [27]: device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        model = model.to(device)

        test_sentences = [
            # Greetings
            "Hello, how are you?",
            "Good morning!",
            "Good night!",
            # Everyday statements
            "I love you.",
            "Thank you very much.",
            "I am hungry.",
            # Moderately complex
            "I am learning artificial intelligence.",
            "The cat is sleeping on the table.",
            "She reads a book every day.",
            "We are going to the park tomorrow.",
            # Technical / academic
            "Artificial intelligence is changing the world.",
            "Deep learning models require a lot of data.",
            "The transformer architecture is very powerful.",
        ]

        trans_start = time.time()
        evaluate_translations(model, test_sentences, eng_word2idx, spa_idx2word,
                             max_len=MAX_SEQ_LEN, device=device)
        print(f"\n13 sentences translated in {time.time() - trans_start:.2f}s")
```

```
=====
                        TRANSLATION RESULTS
=====
[01] EN: Hello, how are you?
      ES: <sos> hola como estas <eos>
-----
[02] EN: Good morning!
      ES: <sos> buenos dias <eos>
-----
[03] EN: Good night!
      ES: <sos> buenas noches <eos>
-----
[04] EN: I love you.
      ES: <sos> te amo <eos>
-----
[05] EN: Thank you very much.
      ES: <sos> muchas gracias <eos>
-----
[06] EN: I am hungry.
      ES: <sos> tengo hambre <eos>
-----
[07] EN: I am learning artificial intelligence.
      ES: <sos> estoy aprendiendo inteligencia artificial <eos>
-----
[08] EN: The cat is sleeping on the table.
      ES: <sos> el gato esta durmiendo en la mesa <eos>
-----
[09] EN: She reads a book every day.
      ES: <sos> ella lee un libro todos los dias <eos>
-----
[10] EN: We are going to the park tomorrow.
      ES: <sos> ma ana vamos al parque <eos>
-----
[11] EN: Artificial intelligence is changing the world.
      ES: <sos> la inteligencia artificial esta cambiando el mundo <eos>
-----
[12] EN: Deep learning models require a lot of data.
      ES: <sos> aprender varios datos requieren mucho aprendizaje <eos>
-----
[13] EN: The transformer architecture is very powerful.
      ES: <sos> la arquitectura es muy poderosa <eos>
-----

13 sentences translated in 0.73s
```

10. Analysis and Limitations

Observations after 10 epochs

Sentence type	Expected quality
Short, high-frequency phrases	Good — these appear many times in the dataset

Sentence type	Expected quality
Medium sentences with common vocabulary	Fair
Long sentences or rare words	Poor

Known limitations

1. **No diacritics in output** — preprocessing removes accents, so translations also lack them.
2. **Greedy decoding** — local token-by-token maximization does not guarantee the best full sequence.
3. **Flat learning rate** — the original paper uses a warm-up + inverse-sqrt decay schedule, which converges better.

How to improve

Technique	Benefit
More training epochs (20–50)	Lower loss, better generalization
Beam search (k = 4–10)	More fluent and accurate translations
BPE / SentencePiece tokenization	Handles morphology and rare words better
LR warm-up + inverse-sqrt decay	Faster and more stable convergence
Label smoothing ($\epsilon = 0.1$)	Reduces overconfidence, improves BLEU

11. Conclusions

Through this activity, we successfully implemented a Transformer-based translator from scratch. We observed that the Attention mechanism is fundamentally superior to previous sequential models because it allows the network to capture long-range dependencies in text without the "vanishing gradient" problem common in RNNs.

translation using the Tatoeba dataset (`eng-spa4.txt`).

Component	Role in the model
Positional Encoding	Injects word-order information without recurrence
Multi-Head Attention	Each token attends to all others; heads capture different relationship types
Encoder	Reads English and produces contextual representations
Decoder	Generates Spanish by attending to previous outputs and the encoder

Component	Role in the model
Masks	Prevent attending to padding and future positions
Teacher Forcing	Feeds gold tokens as decoder input for stable training
Greedy Decoding	Generates one token at a time by picking the highest-probability next token

The Transformer is the backbone of virtually every modern language model — GPT, BERT, T5, LLaMA, Claude, and beyond.

Understanding its implementation from first principles is a foundational skill in deep learning.